Deevolight

Generic Transaction Screens - V2 Architecture

Project overview, database metadata, runtime architecture and integration with Generic Report Writer V1

Architecture reference - October 2026

1. Project Context

Deevolight already has a working Generic Report Writer V1. V2 extends that platform with metadata-driven transaction/data-entry screens and report navigation. The goal is not to replace the V1 report engine. Instead, V2 adds a reusable transaction-screen framework that can open from reports, support Add/View/Edit/Delete, use reusable lookups, apply tenant presentation overrides, and enforce user permissions.

- Backend stack: Python, FastAPI, SQLAlchemy 2.x/Pydantic v2, SQL Server 2025.
- Frontend stack: React, TypeScript, Vite, Tailwind and shadcn/ui.
- V1 reports remain stored-procedure driven and continue using CriteriaJson and MaxRows.
- Transaction CRUD is entity-specific: LeadService, CustomerService, etc. Metadata never generates arbitrary SQL.
- One generic React transaction renderer is reused across entities; metadata controls layout and field behavior.
- Lead should be the first end-to-end proof transaction.

2. Design Goals

- Define a transaction screen once in metadata and render it consistently in React.
- Keep business logic in entity-specific backend services rather than database metadata.
- Allow Deevolight to define system fields and tenants to rename/reorder/show approved optional fields.
- Use reusable lookup definitions for Customer, Style, Vendor, Payment Terms and similar selections.
- Support report-to-screen navigation and report-to-report drilldown without changing the V1 report execution model.
- Start permission rollout in open mode and move individual reports/screens to restricted mode when assignments are ready.

3. Architectural Boundaries

```
Generic Report Writer V1
    |
    +-- Report grid / criteria / saved views
    +-- Report stored procedure execution
    +-- V2 report actions
             |
             +--> another report
             |
             +--> Generic Transaction Screen
                       |
                       +--> screen metadata API
                       +--> entity-specific REST API
                       +--> lookup APIs
                       +--> permission resolver
                       |
                       +--> SQLAlchemy entity service
                                |
                                +--> SQL Server
```

The report framework launches a target and passes context. The target report or transaction screen owns its own security, data loading and business behavior.

4. Database Metadata Model

Table	Responsibility
txn.screen	System-level transaction screen definition; API base path, layout and screen capabilities.
txn.screen_section	Groups fields into vertically stacked sections or tabs.
txn.screen_field	Defines API field key, control, type, order, validation, edit behavior and optional lookup.

Table	Responsibility
txn.screen_field_config	Tenant presentation overrides: display name, order and visibility only.
txn.lookup	Reusable lookup definition and lookup API endpoint.
txn.lookup_column	Columns displayed/searched in a lookup popup.
txn.screen_permission	User-level access and CRUD permissions for a screen.
rpt.report_permission	User-level access to a report.
rpt.report_action	Defines toolbar/row/column actions from a report to a report or screen.
rpt.report_action_param	Maps source row/criteria values into target context.

4.1 txn.screen

A screen is a system-level definition. It has no tenant_id. Important fields include screen_key, display_name, api_base_path, layout_type, permission_mode and allow_view/add/edit/delete.

```
Example:
screen_key       = lead
api_base_path    = /api/v1/leads
layout_type      = standard
permission_mode  = open
allow_view       = 1
allow_add        = 1
allow_edit       = 1
allow_delete     = 0
```

4.2 Sections and Fields

Every field belongs to a section. A standard layout renders sections vertically; a tabs layout renders the same sections as tabs. screen_field.field_key is the API/Pydantic property name, not a physical database column.

```
Lead Screen
  General
    lead_name       text        editable
    company_name    text        editable
    salesperson_id  dropdown    editable
  Contact
    email           email       editable
    phone           phone       editable
  System
    created_at      datetime    readonly
```

edit_mode has three values: editable, create_only and readonly. View mode makes all fields readonly regardless of edit_mode.

4.3 Tenant Field Configuration

Tenants cannot create arbitrary custom fields. Deevolight can predefine optional fields such as remark1/remark2/remark3. txn.screen_field_config may change only display_name, display_order and is_displayed. Data type, validation, lookup and edit behavior remain system-controlled.

4.4 Reusable Lookups

A dropdown can reference txn.lookup. The lookup defines an API endpoint and one return_field_key. txn.lookup_column defines the popup columns and which columns participate in search.

```
Customer lookup:
API: /api/v1/lookups/customers
Return: customer_id

Popup:
Customer ID | Name | City | Address
C1001       | ABC  | NY   | 10 Main St
```

5. Permission Model

Both reports and transaction screens use permission_mode = open or restricted.

```
OPEN
  no user permission row -> allow according to screen/report defaults
  explicit permission row -> explicit permission applies

RESTRICTED
  no user permission row -> deny
  explicit permission row -> explicit permission applies
```

For transaction screens, the final operation is also limited by screen capability. A user cannot edit if screen.allow_edit = 0 even when screen_permission.can_edit = 1. FastAPI is the security boundary; React permission handling is for user experience only.

6. Generic Transaction Screen Runtime

```
Route: /transactions/lead/{id}?mode=view

React TransactionScreen
        |
        +--> GET screen metadata for "lead"
        |
        +--> GET /api/v1/leads/{id}
        |
        +--> merge metadata + record
        |
        +--> render sections and controls
        |
        +--> user selects Edit
        |
        +--> PATCH /api/v1/leads/{id}
        |
        +--> FastAPI LeadService
        |
        +--> SQLAlchemy / SQL Server
```

The generic renderer does not know the Lead table structure. It knows the screen metadata and receives a normal JSON record from the Lead API.

7. Screen Modes

Mode	Behavior
add	No existing record required. Apply metadata defaults. Editable/create_only fields may be entered. POST on Save.
view	Load existing record. All fields readonly. Show Edit/Delete actions only when effective permissions allow.
edit	Load existing record. editable fields can change; create_only and readonly remain readonly. PATCH on Save.

8. Report Integration

8.1 Report to Transaction Screen

```
Lead List report
  -> click lead_name
  -> rpt.report_action
  -> target_screen = lead
  -> rpt.report_action_param: lead_id -> id
  -> open Lead transaction screen in view mode
```

8.2 Report to Report

```
Sales Summary
  -> click sales_amount
  -> action params resolve:
       row.customer_id
       criteria.from_date
       criteria.to_date
```

```
-> build normal CriteriaJson
-> execute Sales Detail through existing V1 report engine
```

9. Concurrency, Audit and Soft Delete

- Entity edit responses include row_version in a transport-safe representation.
- PATCH sends the row_version originally read by the client.
- SQLAlchemy update must enforce optimistic concurrency; stale data returns HTTP 409 Conflict.
- Deletes are soft deletes: is_deleted=1, is_active=0, deleted_at/deleted_by populated.
- Application writes audit.audit_log synchronously in the same transaction.
- updated_at and updated_by are maintained on every write.

10. Initial Implementation Scope

- First backend deliverable: generic screen metadata API.
- First frontend deliverable: generic TransactionScreen renderer that consumes metadata.
- First entity proof: Lead.
- Then add Add/View/Edit behavior, one reusable lookup, permissions and row_version handling.
- After the Lead transaction works end-to-end, connect the existing Lead List report to the transaction screen.
- Report-to-report drilldown follows after report-to-screen navigation is proven.
- Administrative metadata-maintenance screens are intentionally deferred.

11. What Is Not Being Built

- No generic dynamic-SQL CRUD engine.
- No entity_key or generic service registry for CRUD routing.
- No arbitrary tenant-created fields.
- No multiselect control in the first version.
- No header/detail child-grid framework until a real transaction requires it.
- No redesign of the working Generic Report Writer V1.